Autoloading and Reloading Constants

This guide documents how constant autoloading and reloading works.

1 Introduction

Ruby on Rails allows applications to be written as if their code was preloaded.

In a normal Ruby program classes need to load their dependencies:

```
require 'application_controller'

require 'post'

class PostsController < ApplicationController
  def index
    @posts = Post.all
  end
end
```

Our Rubyist instinct quickly sees some redundancy in there: If classes were defined in files matching their name, couldn't their loading be automated somehow? We could save scanning the file for dependencies, which is brittle.

Moreover, `Kernel#require` loads files once, but development is much more smooth if code gets refreshed when it changes without restarting the server. It would be nice to be able to use `Kernel#load` in development, and `Kernel#require` in production.
Indeed, those features are provided by Ruby on Rails, where we just write

```
class PostsController < ApplicationController
  def index
    @posts = Post.all
  end
end
```

This guide documents how that works.

2 Constants Refresher

While constants are trivial in most programming languages, they are a rich topic in Ruby.

It is beyond the scope of this guide to document Ruby constants, but we are nevertheless going to highlight a few key topics. Truly grasping the following sections is instrumental to understanding constant autoloading and reloading.

2.1 Nesting

Class and module definitions can be nested to create namespaces:

```
module XML
  class SAXParser
    # (1)
  end
```